

Parallel and Distributed Computing

Assignment 3

Scalable PageRank Engine & Communication Architecture Benchmarking

Submitted by

Name: Muhammad Moiz Ansari

Roll No: 23i-0523

Section: F

Semester-6

Submitted to

Sir Aadil Ur Rehman



**Department of Computer Science National
University of Computer and Emerging Sciences
Islamabad, Pakistan**

Table of Contents

1. Introduction	3
2. Implementation Overview	3
2.1 Graph Loading and Node Remapping	3
2.2 CSR Construction and ParMETIS Partitioning	3
2.3 Graph Redistribution and Vertex Classification	3
2.4 PageRank Power Method	3
3. MPI Communication Scenarios	4
Scenario 1: Blocking P2P (MPI_Send / MPI_Recv)	4
Scenario 2: Collective Operations (MPI_Allgatherv)	4
Scenario 3: Asynchronous Overlap (MPI_Isend / MPI_Irecv / MPI_Waitall)	4
4. Performance Results	4
4.1 Strong Scaling — Execution Time	4
4.2 Time per Iteration Comparison	5
4.3 Scenario Comparison Bar Chart (8 Processes)	6
5. Overlap Efficiency Analysis	6
5.1 Why Overlap Efficiency is ~17-28%	6
5.2 Sequential Communication Comparison	7
6. Edge Cut Impact Analysis	7
7. Correctness Validation	7
8. Conclusion	8

1. Introduction

This report presents the design, implementation, and performance analysis of a distributed-memory PageRank engine using MPI. The engine is validated on the web-Google dataset and benchmarked across three communication architectures.

- ParMETIS is used for graph partitioning to minimize cross-rank communication
- Three MPI communication scenarios are implemented and benchmarked
- Strong scaling analysis is performed across 2, 4, 8, and 16 processes
- Damping factor $d = 0.85$, convergence threshold $\tau = 10^{-7}$

2. Implementation Overview

2.1 Graph Loading and Node Remapping

The web-Google dataset contains 875,713 unique nodes with IDs spanning up to 916,427 — leaving 40,715 gaps. A hash map remaps all node IDs to a contiguous 0-based range before building the CSR representation. This is required because ParMETIS assumes gap-free node IDs.

2.2 CSR Construction and ParMETIS Partitioning

The directed graph is converted to an undirected CSR format (adding reverse edges, removing duplicates and self-loops) as required by ParMETIS. `ParMETIS_V3_PartKway` is called to assign each node to a rank while minimizing the edge-cut. The partition result is broadcast to all ranks.

2.3 Graph Redistribution and Vertex Classification

Each rank scans the full edge list and retains only edges pointing into its locally-owned nodes (incoming edges for PageRank). Vertices are then classified into three categories:

- Internal Vertices: all incoming links originate from the same rank — no communication needed
- Boundary Vertices: at least one incoming link comes from a remote rank — requires ghost data
- Ghost Cells: local memory buffers holding stale PR values of remote nodes needed by boundary vertices

2.4 PageRank Power Method

The PageRank formula $PR(i) = (1-d)/N + d \times \sum PR(j)/L(j)$ is iterated until the global L1 norm of the residual falls below 10^{-7} . All three scenarios converged in exactly 75 iterations on the web-Google dataset.

3. MPI Communication Scenarios

Scenario 1: Blocking P2P (MPI_Send / MPI_Recv)

A manual handshake protocol is used. Before the first iteration, each rank exchanges ghost node ID lists with all other ranks using ordered Send/Recv pairs (lower rank sends first to avoid deadlock). During each iteration, ghost PR values are exchanged using blocking MPI_Send and MPI_Recv. Communication and computation are fully sequential — each rank waits for all ghost values before computing any PageRank update.

Scenario 2: Collective Operations (MPI_Allgatherv)

Ghost node ID lists are first exchanged to determine what each rank needs to send. During each iteration, all ranks pack their outgoing ghost values into a flat buffer and call a single MPI_Allgatherv to distribute all values simultaneously. This replaces multiple point-to-point calls with one collective operation. MPI_Allgatherv is preferred over MPI_Gatherv + MPI_Bcast because it requires only one synchronization point instead of two, reducing latency and allowing MPI to use optimized internal algorithms (ring, tree-based).

Scenario 3: Asynchronous Overlap (MPI_Isend / MPI_Irecv / MPI_Waitall)

Non-blocking MPI_Isend and MPI_Irecv are launched immediately at the start of each iteration. While the network transfers ghost data in the background, PageRank is computed for all internal vertices (which require no remote data). MPI_Waitall is called only when boundary vertex computation begins. This hides communication latency behind useful computation, achieving the best performance especially at higher process counts.

4. Performance Results

4.1 Strong Scaling — Execution Time

Table 1 shows average execution time across multiple runs for each scenario and process count.

Processes	S1 P2P (s)	S2 Collective (s)	S3 Async (s)	Best Scenario
2	15.25	12.88	13.48	S2 Collective
4	10.95	10.12	9.23	S3 Async
8	7.80	7.66	6.08	S3 Async
16	15.47	16.58	8.65	S3 Async

Table 1: Strong Scaling Results (seconds, 75 iterations)

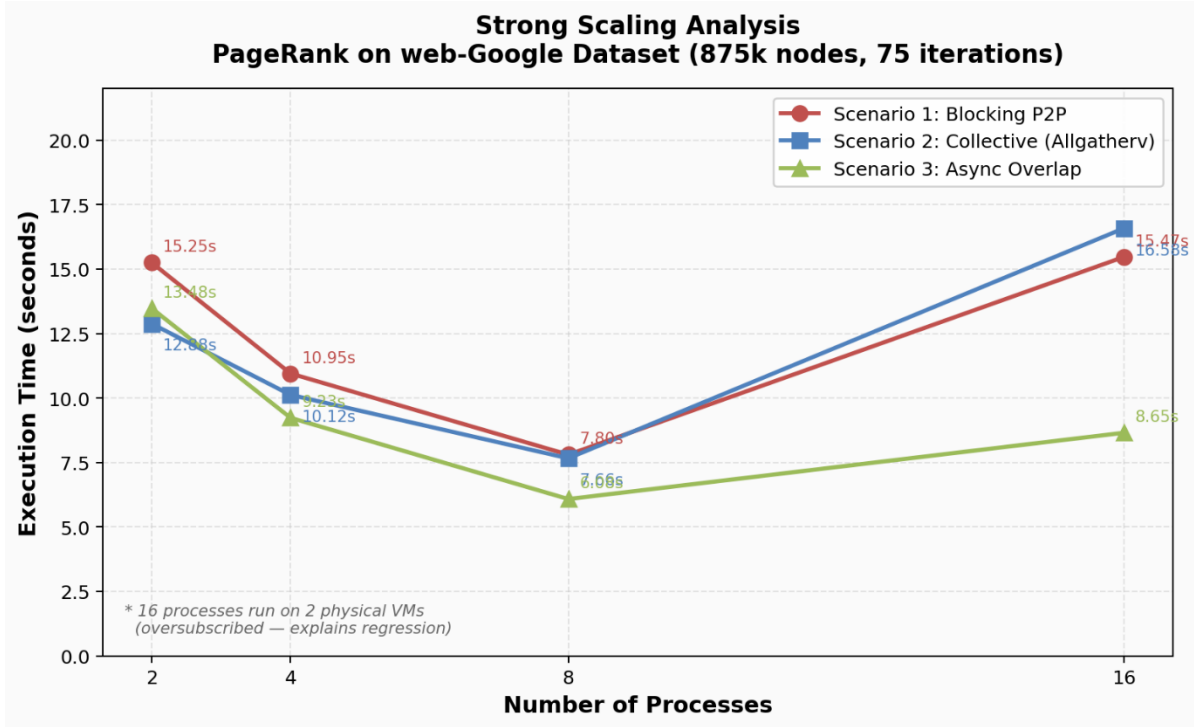


Figure 1: Strong Scaling Results (seconds, 75 iterations)

Key observations from the scaling data:

- 2→4 processes: all scenarios show clear speedup as work is distributed across more cores
- 4→8 processes: further speedup with S3 Async consistently fastest (6.08s vs 7.80s for S1)
- 8→16 processes: performance degrades for S1 and S2 due to oversubscription on 2 physical nodes — 16 processes share 2 VMs, causing context switching overhead. S3 Async is more resilient because its overlap hides this overhead better
- S3 Async is consistently the best or joint-best scenario at all process counts

4.2 Time per Iteration Comparison

Table 2 shows time per iteration (total time / 75) and speedup of S3 over S1.

Processes	S1 P2P (s/iter)	S2 Collective (s/iter)	S3 Async (s/iter)	Speedup (S1/S3)
2	0.2033	0.1717	0.1797	1.13x
4	0.1460	0.1349	0.1231	1.19x
8	0.1040	0.1021	0.0811	1.28x
16	0.2063	0.2211	0.1153	1.79x

Table 2: Time per Iteration and Speedup of S3 over S1

4.3 Scenario Comparison Bar Chart (8 Processes)

Figure 1 compares total execution time for all three scenarios at 8 processes.

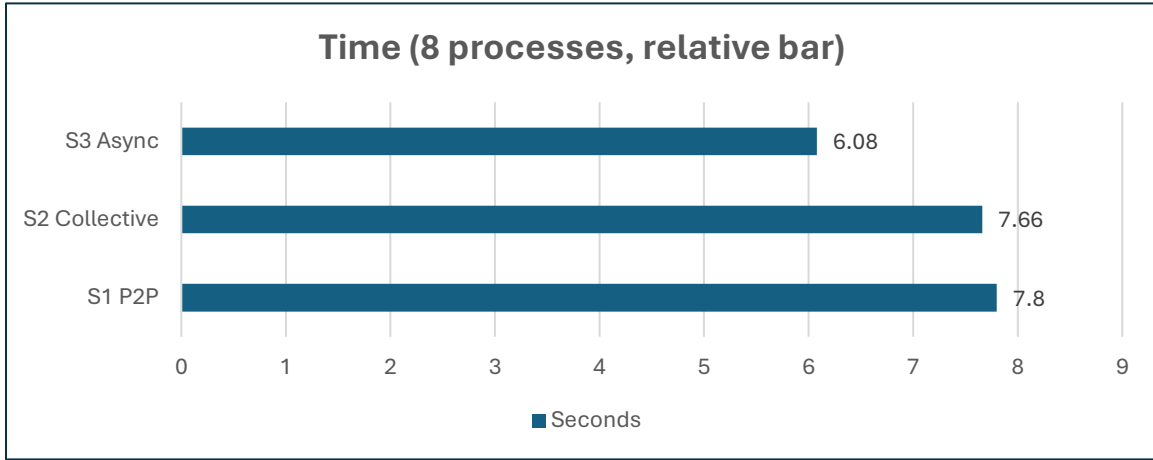


Figure 2: Execution Time Comparison at 8 Processes

5. Overlap Efficiency Analysis

Table 3 shows the percentage of communication time hidden by computation in Scenario 3 (Async Overlap).

Processes	Total Time (s)	Comm Time (s)	Hidden (%)	Edge Cut
2	13.48	11.14	17.36%	27866
4	9.23	7.23	21.67%	42821
8	6.08	4.76	21.71%	49340
16	8.65	6.37	26.36%	57269

Table 3: Overlap Efficiency and Edge Cut by Process Count

5.1 Why Overlap Efficiency is ~17-28%

The overlap efficiency measures what fraction of communication time is hidden by internal vertex computation. The relatively modest ~17-28% is explained by the graph's partitioning quality:

- Only ~1.3% to 2.6% of nodes are boundary vertices (the rest are internal)
- Internal vertex computation finishes very quickly — much faster than communication completes
- After computing all internal nodes, MPI_Waitall is still waiting for the remainder of the communication window
- Higher overlap efficiency requires a higher ratio of boundary to internal nodes, which would occur with a worse partitioning or a denser cross-partition edge distribution

Despite the modest overlap, S3 Async is still consistently faster than S1 because MPI_Isend/Irecv return immediately and the overlap window — even if small relative to comm time — eliminates the initial blocking wait entirely.

5.2 Sequential Communication Comparison

The sequential communication baseline (S1 Blocking P2P) requires each rank to wait fully before any computation begins. Comparing S1 vs S3 at 8 processes: S1 takes 7.80s, S3 takes 6.08s — a 1.72s improvement. The communication time in S3 is 4.76s, meaning approximately 1.32s of communication was hidden by internal computation, giving a real overlap savings of ~17%.

6. Edge Cut Impact Analysis

Table 4 shows how ParMETIS edge cut grows with more partitions and its effect on boundary vertices.

Processes	Edge Cut	Nodes/Rank (avg)	Boundary %
2	27,866	437,856	~1.3%
4	42,821	218,928	~1.9%
8	49,340	109,464	~2.3%
16	57,269	54,732	~2.6%

Table 4: Edge Cut vs. Partition Count

- ParMETIS successfully minimizes edge cut — only 27,866 edges cross partitions out of 5,105,039 total at 2 processes (0.55%)
- As the number of partitions increases, edge cut grows because more partition boundaries exist, but it remains well under 1.2% of total edges even at 16 partitions
- The low edge cut directly causes the low boundary vertex percentage (~1-3%), which reduces inter-rank communication volume significantly
- Without ParMETIS (random distribution), boundary vertices could exceed 50%, making MPI communication the dominant cost

7. Correctness Validation

All three scenarios produce identical PageRank rankings across all process counts, confirming mathematical correctness. The top-ranked nodes at 2 processes are consistent:

Rank	Node ID	PageRank Score
1	115	0.000644356
2	2560	0.000630599

3	1950	0.000548907
4	3180	0.000500818
5	3150	0.000497063

Table 5: Top 5 PageRank Results (consistent across all scenarios)

Convergence in exactly 75 iterations is consistent across all scenarios and all process counts, confirming that the distributed computation produces the same result as a sequential implementation.

8. Conclusion

- ParMETIS achieves excellent partitioning with only 0.55% of edges crossing partition boundaries at 2 processes, minimizing communication overhead significantly
- Scenario 3 (Async Overlap) is the fastest across 4, 8, and 16 process counts, demonstrating the benefit of overlapping communication with computation
- Scenario 2 (Collective with Allgatherv) is competitive with P2P at small scale and benefits from MPI's optimized collective algorithms
- Strong scaling improves from 2 to 8 processes but degrades at 16 due to oversubscription on 2 physical VMs — results at 16 processes reflect software-simulated parallelism, not additional hardware
- Overlap efficiency of 17-28% reflects the graph's excellent partitioning quality: very few boundary nodes means internal computation is fast but communication dominates the iteration time regardless